



Moteur d'Inférence de Réseaux de Neurones pour la Reconnaissance de Chiffres Manuscrits

Auteurs:

Jianye SHI

Responsables:

Mathys E. Jam / Pablo Oliveira

December 15, 2025

Sommaire

1	Introduction	3
2	Environnement Expérimental	3
2.1	Configuration Matérielle	3
2.2	Configuration Logicielle	4
2.3	Protocole de mesure	4
3	Implémentation et amélioration du SGEMM	4
3.1	SGEMM naïf et analyse de performance initiale	4
3.2	Optimisations ikj, blockées et OpenMP	5
3.3	Analyse de performance après optimisation	6
3.4	Impact de la vectorisation	8
3.5	Analyse des événements de cache	8
3.6	Réglages d’affinité OpenMP	9
4	Moteur d’Inférence de Réseaux de Neurones	9
4.1	Architecture générale	9
4.2	Opérateurs supportés	10
4.3	Planificateur d’exécution	10
4.4	Validation et tests	10
5	Analyse de performance du moteur	11
5.1	Les mesures de performance	11
5.2	CLI <code>-gemm</code> / <code>-profile</code> et mesure des opérateurs	11
5.3	Impact des variantes SGEMM sur l’inférence (analyse qualitative)	12
6	Conclusion	12
A	Scripts expérimentaux et protocoles	13
A.1	Protocole de mesure	13
A.2	Balayage des tailles	13
A.3	Comparaison des optimisations	13
A.4	Vectorisation	13
A.5	Événements de cache	13
A.6	Affinité OpenMP	14
A.7	Benchmarking et Profiling	14

1 Introduction

Ce projet s’inscrit dans le cadre de l’optimisation de performance et de l’implémentation de moteurs d’inférence pour réseaux de neurones. Il se compose de deux parties principales : l’optimisation d’un noyau SGEMM (Single-precision General Matrix Multiply) dans le Lab 6, puis l’utilisation de ce noyau optimisé pour implémenter un moteur d’inférence permettant la reconnaissance de chiffres manuscrits du dataset MNIST dans le Lab 7.

Le SGEMM est une opération fondamentale dans le calcul scientifique et l’apprentissage automatique, représentant souvent la majorité du temps de calcul dans les réseaux de neurones. L’optimisation de cette opération permet d’améliorer de manière notable les performances globales des applications. Dans ce travail, nous avons exploré plusieurs techniques d’optimisation : la réorganisation des boucles pour améliorer la localité mémoire ; le découpage en blocs pour maximiser l’utilisation du cache ; et la parallélisation avec OpenMP pour exploiter les architectures multi-cœurs.

Le moteur d’inférence développé dans le Lab 7 permet d’exécuter des modèles de réseaux de neurones au format ONNX, avec support pour quatre opérations de base : Flatten, Add, Relu et Gemm. L’objectif est de reconnaître des chiffres manuscrits avec une précision élevée tout en minimisant le temps d’exécution et la consommation énergétique.

2 Environnement Expérimental

Toutes les expériences ont été réalisées sur une machine avec les spécifications suivantes :

2.1 Configuration Matérielle

Table 1: Configuration matérielle

Composant	Caractéristiques
Processeur	AMD Ryzen 9 8940HX with Radeon Graphics Architecture : x86_64 Cœurs / Threads : 16 physiques / 32 logiques (2 threads par cœur) Fréquence : 421.8 MHz (min) à 5386 MHz (max) Cache L1d : 512 KiB (16 instances) Cache L1i : 512 KiB (16 instances) Cache L2 : 16 MiB (16 instances) Cache L3 : 64 MiB (2 instances)
Mémoire	30 GiB de RAM système
Swap	8.0 GiB

Les expérimentations parallèles ainsi que les mesures d’énergie ont été réalisées sur cette même configuration matérielle.

2.2 Configuration Logicielle

Table 2: Configuration logicielle

Composant	Version / Détails
Système d'exploitation	Fedora Linux 42 (Workstation Edition)
Compilateur C/C++	GCC 15.2.1 20250808 (Red Hat 15.2.1-1)
Python	Version 3.13.7
CMake	Version 3.31.6
Shell	zsh
IDE	Visual Studio Code on Linux

Les mesures de performance ont été réalisées à l'aide de `perf stat` avec les événements RAPL (`power/energy-pkg/` et `power_core/energy-core/`) pour quantifier la consommation énergétique. Les optimisations de compilation ont été testées avec les flags `-O3 -march=native` pour activer l'auto-vectorisation et les optimisations spécifiques à l'architecture.

2.3 Protocole de mesure

Afin de limiter la variabilité des résultats, les campagnes de mesure suivent un protocole systématique de verrouillage de fréquence et d'affinité CPU. Les huit premiers cœurs physiques sont forcés à 4.0 GHz à l'aide de `cpupower`, puis les exécutions `perf stat` sont liées explicitement à ce jeu de cœurs via `taskset`. Le script type est le suivant : Le détail des commandes (verrouillage de fréquence, événements suivi par `perf stat`, étapes de remise à l'état initial) est présenté en annexe A.1. Ce protocole constitue l'adaptation AMD des commandes recommandées pour les processeurs Intel dans le Lab 6, lesquelles préconisent un verrouillage similaire sur quatre cœurs logiques avec des bornes de fréquence adaptées au turbo (par exemple `cpupower frequency-set -g performance -u 5000MHz -d 5000MHz`). Les deux approches ne diffèrent donc que par les limites matérielles et la taille du groupe de cœurs réservés à l'expérience. Les scripts d'analyse acceptent les variables `LOCK_FREQ` et `CPU_SET` afin de rejouer automatiquement ces réglages.

3 Implémentation et amélioration du SGEMM

3.1 SGEMM naïf et analyse de performance initiale

L'opération SGEMM calcule $RES = A \times B + C$, où A est une matrice $M \times K$, B est une matrice $K \times N$, et C et RES sont des matrices $M \times N$. La version de base suit simplement trois boucles imbriquées :

```
for each row i of A:
    for each column j of B:
        sum = dot(A[i, :], B[:, j])
        RES[i, j] = sum + C[i, j]
```

Cette implémentation naïve présente un problème de performance. Dans la boucle interne sur k , l'accès $B[k*N+j]$ parcourt B avec un saut de N éléments, traversant ainsi la matrice par colonnes (stride N) et générant de nombreux défauts de cache sur une organisation row-major.

Pour caractériser ce comportement, nous avons exécuté `sgemm naive` avec $N = K = 512$ et M croissant, en verrouillant la fréquence des cœurs selon le protocole décrit ci-dessus (Section 2.3). La figure 1 met en évidence une croissance quasi linéaire du temps d'exécution et de l'énergie consommée, ce qui est cohérent avec la complexité $O(MNK)$ lorsque deux dimensions sont fixées.

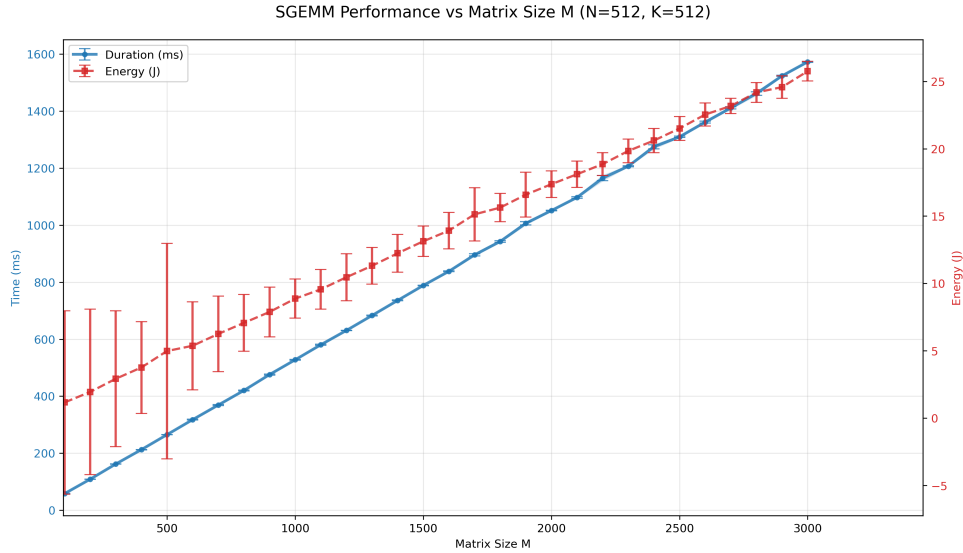


Figure 1: Impact de la taille de matrice sur les performances (version naïve)

Les barres d'erreur de l'énergie diminuent lorsque la taille de la matrice augmente, car les exécutions plus longues rendent la mesure RAPL plus stable et moins sensible aux variations instantanées de la puissance CPU.

Ces résultats illustrent la tendance de la version naïve ; nous l'utiliserons donc comme point de départ pour les optimisations qui suivent.

3.2 Optimisations *ikj*, bloquées et OpenMP

Les optimisations présentées ci-dessous visent à rapprocher le noyau SGEMM de l'idéal mémoire du processeur : la réorganisation *ikj* améliore la localité en parcourant séquentiellement les lignes de B pour limiter les défauts de cache ; le découpage en blocs confine les données actives dans les niveaux rapides de la hiérarchie ; la parallélisation OpenMP étale enfin la charge restante sur plusieurs cœurs pour masquer les latences.

Réorganisation *ikj*. Nous réorganisons les boucles dans l'ordre (i, k, j) de façon à parcourir A et B ligne par ligne. Concrètement, on initialise RES avec C , puis pour chaque couple (i, k) on additionne la ligne $B[k, :]$ pondérée par $A[i, k]$. Cette structure évite les accès en colonne dans B et réduit drastiquement les défauts de cache, d'où le nom de variante `sgemm_ikj`. L'usage systématique des pointeurs `restrict` indique au compilateur l'absence d'alias, ce qui favorise l'auto-vectorisation.

Découpage en blocs. La variante `sgemm_blocked` découpe les trois boucles selon un pas de 128, de sorte que chaque itération travaille sur un sous-ensemble borné des matrices A , B et RES . Les données actives demeurant confinées à ces sous-régions, les accès traversent moins de lignes de cache ; la hiérarchie matérielle assure alors une meilleure réutilisation des blocs en L1/L2, ce qui améliore la localité et réduit le taux de défauts.

Parallélisation OpenMP. À partir du noyau *ikj*, nous parallélisons la boucle externe sur les blocs (ou lignes) de RES au moyen d'OpenMP. Le planificateur `schedule(static)` assure que chaque thread reçoit le même nombre de blocs, ce qui offre un partage de charge efficace pour des matrices régulières.

Optimisations de compilation. Enfin, les options `-O3 -march=native` activent l’auto-vectorisation et l’utilisation des instructions SIMD spécifiques à l’architecture (AVX2/AVX-512 sur notre machine), complétant les gains obtenus par les transformations algorithmiques précédentes.

3.3 Analyse de performance après optimisation

Les mesures de performance ont été effectuées à l’aide de `perf stat` afin d’observer l’impact des optimisations sur le temps, l’énergie et les compteurs matériels. Nous utilisons la version monothread d’OpenBLAS comme cible idéale à atteindre sur CPU. L’ensemble des résultats présentés ci-dessous suit le protocole expérimental décrit en Section 2.3.

La mesure de référence est résumée dans le tableau 3 : elle porte sur 20 répétitions de `sgemm naive 512 512 512` avec les mêmes réglages d’affinité et de fréquence.

Table 3: Mesure de référence pour `sgemm naive 512 512 512`

Test	Répétitions	Temps moyen (s)	Énergie pkg (J)
<code>sgemm naive 512 512 512</code>	20	0.27599	10.26

Deux scripts automatisent ensuite le balayage de M (avec $N = K = 512$) :

- `performance/compare-sizes.sh`, qui produit le CSV `performance/sgemm_sizes_comparison.csv` ;
- `performance/plot-sizes.py`, qui transforme ce CSV en figure 1.

Le script `performance/compare-optimizations.sh` évalue `naive`, `ikj`, `blocked`, `omp` (1–16 threads) et `blas` pour $M \in \{512, 1024, 2048, 4096\}$. Les cycles, instructions et mesures RAPL sont stockés dans un CSV long, puis `performance/plot-optimizations.py` synthétise directement les distributions de temps et d’énergie (notamment pour $M = 512$ et $M = 4096$) qui alimentent la figure 2.

La figure 2 illustre la comparaison détaillée des différentes versions optimisées pour $M = 512$ et $M = 4096$, mettant en parallèle les effets sur le temps et l’énergie.

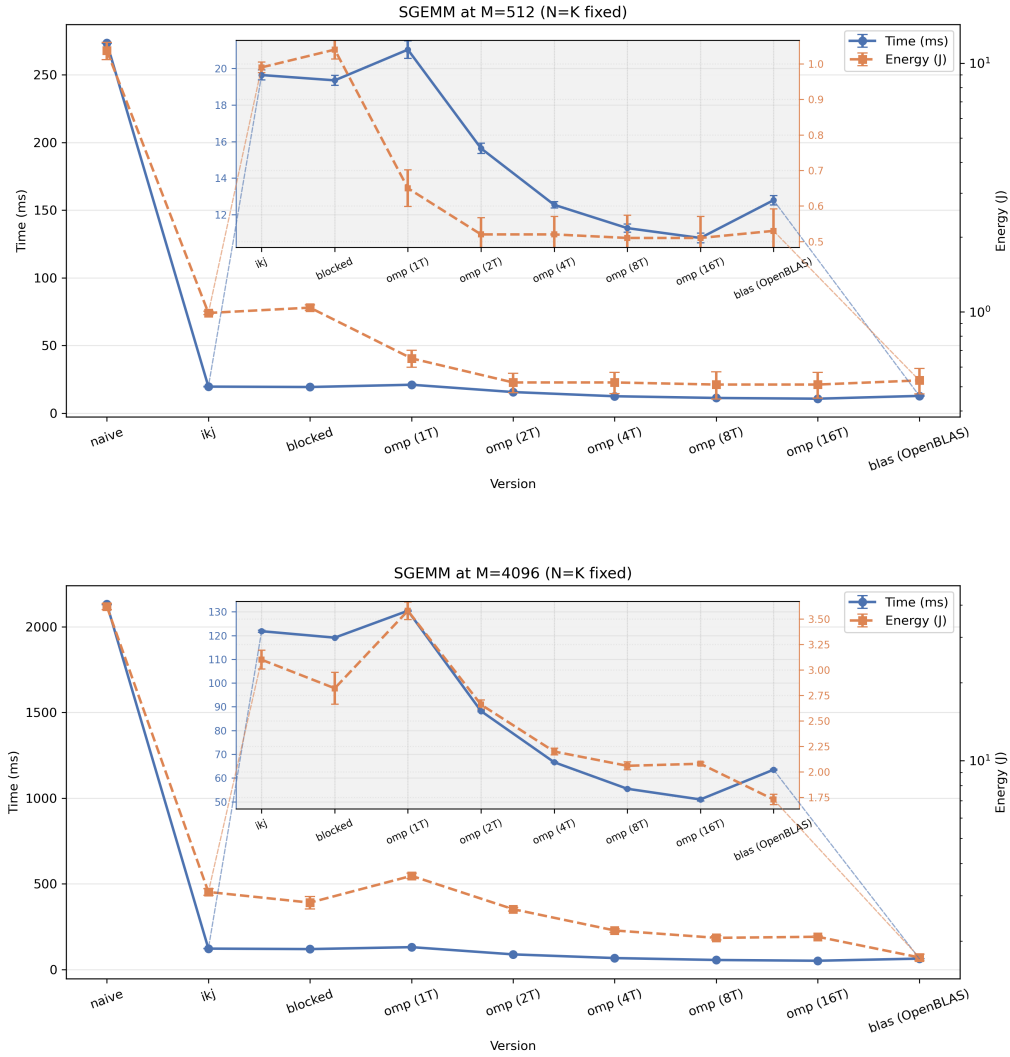


Figure 2: Comparaison des performances et de la consommation d'énergie des différentes implémentations de SGEMM pour deux tailles de matrice

Observations clés. Plusieurs enseignements ressortent de ces campagnes :

- **Localité mémoire** : la variante ikj apporte le gain le plus significatif en supprimant les accès discontinus.
- **Parallélisation** : l'ajout de threads OpenMP réduit fortement le temps d'exécution jusqu'à 8 threads, puis le gain se tasse ; pour $M = 4096$, passer de 8 à 16 threads divise encore presque par deux le temps mais avec un rendement décroissant.
- **Consommation énergétique** : l'énergie chute nettement entre 1 et 8 threads, puis se stabilise ; les configurations 8T et 16T affichent des consommations proches alors que la seconde reste plus rapide.

Comparaison des versions optimisées. Le tableau 4 présente les résultats de performance pour différentes implémentations sur une matrice $512 \times 512 \times 512$. La version ikj apporte une accélération d'environ 14x par rapport à la version naïve, principalement grâce à l'amélioration de la localité mémoire. La version blocked offre des performances similaires, avec une légère

amélioration pour les grandes matrices. La version OpenMP atteint environ 24x à 8 threads et 25x à 16 threads, les gains devenant marginaux au-delà.

Table 4: Comparaison des performances SGEMM (M=512, N=512, K=512)

Version	Temps (ms)	Énergie (J)	Accélération
naive	273.6	11.25	1.0x
ikj	19.6	0.99	14.0x
blocked	19.4	1.04	14.1x
omp (8 threads)	11.3	0.51	24.2x
omp (16 threads)	10.7	0.51	25.5x
blas	12.8	0.53	21.4x

Des campagnes complémentaires ont été menées pour caractériser l’impact de la vectorisation, du comportement des caches et de l’affinité OpenMP.

3.4 Impact de la vectorisation

Le binaire `build-o3` est compilé avec `-O3 -march=native -fopt-info-vec-optimized`. Le journal `performance/vectorization_build.log` atteste que les noyaux `naive`, `ikj`, `blocked` et `omp` disposent de boucles vectorisées (largeurs 64B/32B/16B). Les campagnes de mesure `perf`, comparant `build/sgemm` et `build-o3/sgemm` donnent les tendances résumées dans le tableau 5. Les gains sont les plus marqués en mono-thread (jusqu’à 1.30× sur `naive`), puis décroissent à mesure que la mémoire devient le facteur limitant.

Table 5: Effet de la vectorisation (`performance/sgemm_perf_default.csv` vs `performance/sgemm_perf_vectorized.csv`)

Implémentation	Accélération moyenne	Variation d’énergie
<code>naive</code> (1 thread)	$\approx 1.30\times$	−20% à −35%
<code>ikj</code> (1 thread)	1.28–1.40×	−15% à −25%
<code>blocked</code> / <code>blas</code>	1.05–1.18×	−10% à −20%
<code>omp</code> (2–4 threads)	1.20–1.45×	−10% à −18%
<code>omp</code> (8+ threads)	$\leq 1.10\times$	baisse modérée, bénéfice limité par le débit mémoire

Au-delà de huit threads, l’accès mémoire devient prépondérant : la vectorisation conserve un léger avantage énergétique mais ne change plus significativement le temps d’exécution.

3.5 Analyse des événements de cache

Les mesures `perf stat` sur $M = 1280$, $N = K = 512$ révèlent l’effet cumulé du réordonnement et du blocage de boucle (tableau 6). Les colonnes listent les compteurs AMD L3 et les défauts L1.

Table 6: Comptage des événements de cache (extrait `performance/sgemm_cache`)

Version	L3 cohérents	L3 hits	L3 miss	L1 miss	Temps (s)
<code>naive</code>	2.79×10^7	2.64×10^7	1.58×10^6	3.76×10^8	0.6744
<code>ikj</code>	1.35×10^7	1.21×10^7	1.33×10^6	2.73×10^7	0.0431
<code>blocked</code>	4.28×10^6	3.05×10^6	1.22×10^6	3.67×10^7	0.0430

Le passage à `ikj` divise par deux les accès L3 cohérents et réduit d'un ordre de grandeur les défauts L1. La version `blocked` se concentre sur des sous-blocs: elle abaisse encore la pression sur le L3 tout en maintenant un temps similaire à `ikj`.

3.6 Réglages d'affinité OpenMP

Pour $M = 2048$, $N = K = 512$ et huit threads, `performance/experiment-omp-affinity.sh` compare plusieurs réglages d'affinité (tableau 7). Les mesures confirment que le placement homogène sur les cœurs physiques est préférable.

Table 7: Impact des stratégies OpenMP (8 threads)

Configuration	Temps (ms)	Énergie pkg (J)
Défaut (auto)	25.11	1.65
OMP_PLACES=cores, PROC_BIND=close	24.95	1.61
OMP_PLACES=cores, PROC_BIND=spread	25.02	1.64
Empilement forcé (threads sur mêmes cœurs)	42.00	2.24

Contraindre plusieurs threads sur un même cœur logique dégrade simultanément temps et énergie.

Ces optimisations constituent le socle logiciel sur lequel s'appuie le moteur d'inférence présenté dans la section 4.

4 Moteur d'Inférence de Réseaux de Neurones

Le moteur d'inférence résultant du Lab 7 exécute des modèles ONNX générés par PyTorch. Il réutilise le noyau SGEMM optimisé pour accélérer les couches `Gemm`, ce qui en fait l'élément central des performances de bout en bout.

4.1 Architecture générale

Pour clarifier la répartition des responsabilités logicielles, le tableau 8 résume les fichiers centraux du moteur et les points remarquables à retenir pour l'analyse de performance.

Table 8: Synthèse des principaux fichiers du moteur d'inférence

Fichier	Rôle principal	Points notables
<code>src/main.c</code>	Interface CLI et chargement du modèle	Options <code>-gemm</code> , <code>-profile</code> , gestion des chemins ONNX
<code>src/inference.c</code>	Orchestration des opérateurs	Implémentations de Flatten, Relu, Gemm et Add, sélection du backend SGEMM
<code>src/inference.h</code>	Structures de données partagées	Définit <code>matrix_t</code> , signatures publiques de l'API inference
<code>src/parse_model.c</code>	Chargement et description du graphe	Parcours des noeuds ONNX, création de <code>value_map</code>
<code>external/sgemm/sgemm.c</code>	Noyaux de multiplication optimisés	Variantes <code>naive</code> , <code>ikj</code> , <code>blocked</code> et <code>omp</code> avec <code>schedule(static)</code>

Représentation des données. Les tenseurs sont encapsulés dans la structure `matrix_t` (dimensions, indicateur `weights`, pointeur sur les données). Les poids sont chargés une seule fois depuis le fichier ONNX et conservés en mémoire row-major pour rester compatibles avec l'ensemble des backends SGEMM (`sgemm_ikj`, `sgemm_omp`, etc.).

Graphe de calcul. Le modèle ONNX est traité comme un graphe acyclique dirigé : chaque nœud représente un opérateur et les arêtes matérialisent le flux de tenseurs. Le parseur (basé sur `onnx-ml.pb-c`) expose des fonctions de navigation permettant de récupérer les entrées, sorties et initialisateurs de chaque nœud.

4.2 Opérateurs supportés

Flatten. Conversion d'un tenseur (N, M) en vecteur colonne $(NM, 1)$ via une simple copie séquentielle.

Add. Addition élément par élément de deux matrices de même forme, opération linéaire aisément vectorisable.

Relu. Application de $\max(0, x)$ sur chaque valeur ; le compilateur vectorise cette boucle simple.

Gemm. Calcul de $Y = W \times X + B$ en s'appuyant sur le backend SGEMM sélectionné à l'exécution. Nous utilisons `sgemm_ikj` par défaut, `sgemm_omp` pour les campagnes multi-thread, tandis que `sgemm_blocked` et `sgemm_blas` servent de points de comparaison. Cette intégration directe assure que les optimisations mémoire (ikj, blocage, parallélisation OpenMP) sont réutilisées dans le moteur.

4.3 Planificateur d'exécution

Le moteur maintient une table de correspondance (`value_map`) associant chaque nom de tenseur à sa matrice courante.

Gestion mémoire. Chaque entrée précise si le tenseur est possédé par le moteur (allocation interne) ou emprunté au graphe (poids). À la fin de l'exécution, les buffers temporaires sont libérés automatiquement, ce qui évite les fuites tout en réutilisant les poids.

Ordonnancement. Les nœuds sont parcourus dans l'ordre topologique fourni par ONNX ; un nœud n'est évalué que lorsque l'ensemble de ses entrées figure dans `value_map`. Cette stratégie simple suffit puisque les modèles étudiés ne contiennent pas de branchement conditionnel.

4.4 Validation et tests

Tests unitaires. Chaque opérateur possède des tests basés sur Unity qui couvrent différentes dimensions et vérifient la précision numérique avec une marge d'erreur serrée.

Validation MNIST. Nous avons exécuté 100 images (`image_0.ubyte` à `image_99.ubyte`) sur trois modèles ONNX de complexité croissante, en forçant toutes les passes à utiliser l'implémentation multi-thread `sgemm_omp` (`OMP_NUM_THREADS=16`). Les résultats sont présentés dans le tableau 10.

Table 9: Résultats de reconnaissance sur le dataset MNIST

Modèle	Précision	Temps (ms)	Énergie pkg (J)
mnist.onnx	98%	72,8	0,41
mnist2.onnx	97%	70,0	0,33
mnist_big.onnx	98%	103,9	1,48

Tous les modèles atteignent une précision supérieure à 97%, démontrant la correctitude de l'implémentation. Le modèle `mnist_big.onnx` reste le plus coûteux à cause de sa profondeur accrue. Les mesures RAPL activées pendant la campagne multi-thread indiquent une dépense moyenne par image comprise entre 0,33 et 1,48 J sur le package CPU.

5 Analyse de performance du moteur

5.1 Les mesures de performance

Les mesures rassemblées au tableau 10 mettent en évidence le lien direct entre la taille du réseau et le temps d'inférence. En configuration `sgemm_omp` (16 threads), le moteur traite une image MNIST en 72,8 ms pour `mnist.onnx` et 70,0 ms pour `mnist2.onnx`, tandis que le modèle plus profond `mnist_big.onnx` requiert 103,9 ms. La consommation énergétique moyenne par image passe simultanément de 0,33 J à 1,48 J selon la complexité du modèle.

5.2 CLI `-gemm / -profile` et mesure des opérateurs

Le point d'entrée `main.c` accepte désormais l'option `-gemm=<backend>` qui sélectionne dynamiquement l'implémentation SGEMM à l'exécution (`ikj`, `blocked`, `omp`, etc.). L'appel est relayé à `inference_set_gemm_backend()` avant le chargement du modèle, ce qui permet de comparer plusieurs variantes sans recompiler.

L'option `-profile` instruit `src/inference.c` pour chronométrer chaque appel à `Flatten`, `Add`, `Relu` et `Gemm` avant d'émettre un bloc `PROFILE_JSON`. Les campagnes automatisées activent les variables d'environnement `PROFILE_INFERENCE=1`, `MEASURE_ENERGY=1` et `REPEAT=1`, fixent `GEMM_BACKEND=omp` avec `OMP_NUM_THREADS=16` et laissent `CPU_SET` non contraint. Les traces consolidées dans `performance/results.json` sont ensuite synthétisées par

```
performance/profile_charts.py -input performance/results.json -output
performance/charts
```

pour produire les graphiques de la figure 3. Sur ce lot de mesures, `Gemm` concentre 83,8 % du temps total pour `mnist` et 79,4 % pour `mnist2`, mais seulement 32,1 % pour `mnist_big`. Le temps non attribué correspond aux copies de tenseurs, aux allocations intermédiaires et aux synchronisations OpenMP, tandis que `Relu` et `Add` restent chacun sous la barre du pourcent.

Table 10: Profils temporels et nombre d'appels par opérateur GEMM pour les modèles `mnist`, `mnist2` et `mnist_big`

Modèle	Temps (ms)	Pourcentage du temps total	Nombre d'appels
mnist.onnx	389,67	83,8	700
mnist2.onnx	260,04	79,4	500
mnist_big.onnx	363,41	32,1	700

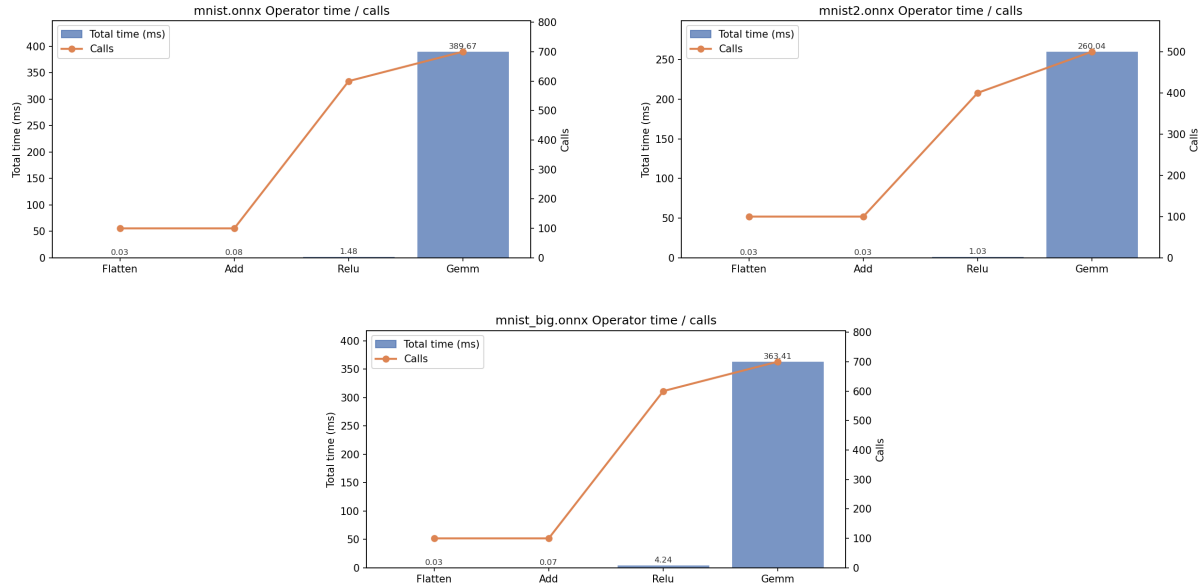


Figure 3: Profils temporels et nombre d'appels par opérateur pour les modèles `mnist`, `mnist2` et `mnist_big`

5.3 Impact des variantes SGEMM sur l'inférence (analyse qualitative)

Les campagnes actuelles n'évaluent que l'implémentation multi-thread `sgemm_omp`. Toutefois, les résultats du Lab 6 indiquent qu'une version naïve allongerait d'un facteur 24 le temps des couches Gemm, rendant l'inférence globale prohibitive. À l'inverse, des variantes mono-thread optimisées (`sgemm_ikj`) ou vectorisées promettent encore des gains importants, notamment pour `mnist_big.onnx`.

6 Conclusion

Ce projet a permis de développer et d'optimiser un moteur d'inférence de réseaux de neurones pour la reconnaissance de chiffres manuscrits. Les principaux résultats sont :

- L'optimisation du noyau SGEMM a permis d'atteindre une accélération de 24x par rapport à l'implémentation naïve, principalement grâce à l'amélioration de la localité mémoire et la parallélisation.
- Le moteur d'inférence atteint une précision de 97-98% sur le dataset MNIST, démontrant la correctitude de l'implémentation.
- Les mesures de performance indiquent des temps d'inférence compris entre 70 et 104 ms par image avec le backend `sgemm_omp` (16 threads) dans les cœurs 0-7 et fréquence donnée par le CPU, pour une énergie moyenne par image allant de 0,33 à 1,48 J sur le package.
- L'analyse révèle que les opérations Gemm restent prépondérantes sur `mnist` et `mnist2` (79–84 % du temps total), mais ne représentent que 32 % sur `mnist_big`, le reste provenant des coûts de gestion multi-thread.

Les travaux futurs pourraient explorer l'optimisation SIMD explicite, la fusion d'opérateurs, et l'inférence par lots pour améliorer encore les performances et l'efficacité énergétique.

A Scripts expérimentaux et protocoles

A.1 Protocole de mesure

```
sudo cpupower frequency-set -g performance -u 4000MHz -d 4000MHz
# power/energy-pkg/ : total package energy
# power_core/energy-core/ : core execution energy
# USE_SUDO=1 : pour forcer l'usage de sudo dans les scripts
USE_SUDO=1 sudo taskset -c 0-7 perf stat -r 20 -a -C 0-7 \
    -e duration_time,power/energy-pkg/,power_core/energy-core/ \
    ./build/sgemm naive 512 512 512

sudo cpupower frequency-set -g powersave
sudo cpupower frequency-set -d 421MHz -u 5386MHz
```

A.2 Balayage des tailles

```
cd performance
./compare-sizes.sh
python3 plot-sizes.py sgemm_sizes_comparison.csv performance_naive.png
```

A.3 Comparaison des optimisations

```
cd performance
sudo cpupower frequency-set -g performance -u 4000MHz -d 4000MHz
USE_SUDO=1 ./compare-optimizations.sh
sudo cpupower frequency-set -g powersave
sudo cpupower frequency-set -d 421MHz -u 5386MHz
python3 plot-optimizations.py --csv sgemm_optimizations_comparison.csv \
    --out-dir . --prefix performance_optimizations
```

A.4 Vectorisation

```
cmake -S . -B build-o3 -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_C_FLAGS="-O3 -march=native -fopenmp -fcommon -fcommon -fcommon"
cmake --build build-o3 --clean-first

cd performance
USE_SUDO=1 ./compare-optimizations.sh
python3 plot-optimizations.py --csv sgemm_perf_vectorized.csv \
    --out-dir . --prefix performance_optimizations_vectorized
```

A.5 Événements de cache

```
sudo cpupower frequency-set -g performance -u 4000MHz -d 4000MHz
# amd_l3/l3_lookup_state.all_coherent_accesses_to_l3/ : requetes totales vers L3
# amd_l3/l3_lookup_state.l3_hit/ : requetes satisfaites en L3
# amd_l3/l3_lookup_state.l3_miss/ : requetes qui manquent L3
# L1-dcache-load-misses : defauts de chargement L1D
sudo taskset -c 0-7 perf stat -a -C 0-7 \
    -e amd_l3/l3_lookup_state.all_coherent_accesses_to_l3/, \
    amd_l3/l3_lookup_state.l3_hit/,amd_l3/l3_lookup_state.l3_miss/, \
    L1-dcache-load-misses \
    ./build/sgemm naive 1280 512 512

sudo taskset -c 0-7 perf stat -a -C 0-7 \
```

```

-e amd_l3/l3_lookup_state.all_coherent_accesses_to_l3/, \
amd_l3/l3_lookup_state.l3_hit/, amd_l3/l3_lookup_state.l3_miss/, \
L1-dcache-load-misses \
./build/sgemm ikj 1280 512 512

sudo taskset -c 0-7 perf stat -a -C 0-7 \
-e amd_l3/l3_lookup_state.all_coherent_accesses_to_l3/, \
amd_l3/l3_lookup_state.l3_hit/, amd_l3/l3_lookup_state.l3_miss/, \
L1-dcache-load-misses \
./build/sgemm blocked 1280 512 512

sudo cpupower frequency-set -g powersave
sudo cpupower frequency-set -d 421MHz -u 5386MHz

```

A.6 Affinité OpenMP

```

sudo cpupower frequency-set -g performance -u 4000MHz -d 4000MHz
USE_SUDO=1 CORE_SET=0-7 OMP_THREADS=8 ./performance/experiment-omp-affinity.sh

OMP_NUM_THREADS=8 OMP_PLACES=cores OMP_PROC_BIND=close \
taskset -c 0-7 perf stat -r 20 -e duration_time,power/energy-pkg/ \
./build/sgemm omp 2048 512 512

OMP_NUM_THREADS=8 OMP_PLACES=cores OMP_PROC_BIND=spread \
taskset -c 0-7 perf stat -r 20 -e duration_time,power/energy-pkg/ \
./build/sgemm omp 2048 512 512
sudo cpupower frequency-set -g powersave
sudo cpupower frequency-set -d 421MHz -u 5386MHz

```

A.7 Benchmarking et Profiling

```

sudo cpupower frequency-set -g performance -u 4000MHz -d 4000MHz
USE_SUDO=1 CORE_SET=0-7 OMP_THREADS=8 ./performance/experiment-omp-affinity.sh

OMP_NUM_THREADS=8 OMP_PLACES=cores OMP_PROC_BIND=close \
taskset -c 0-7 perf stat -r 20 -e duration_time,power/energy-pkg/ \
./build/sgemm omp 2048 512 512

OMP_NUM_THREADS=8 OMP_PLACES=cores OMP_PROC_BIND=spread \
taskset -c 0-7 perf stat -r 20 -e duration_time,power/energy-pkg/ \
./build/sgemm omp 2048 512 512
sudo cpupower frequency-set -g powersave
sudo cpupower frequency-set -d 421MHz -u 5386MHz

```